gc_content.py 사용 설명서

1. 개요

src/gc_content.py 는 FASTQ 파일의 **각 read 마다 GC 비율(%)과 평균 품질 점수**를 함께 계산하는 스크립트이다.

- read 별 결과를 이름<TAB>길이<TAB>GC(%)<TAB>평균품질 형식의 표로 출력한다.
- 마지막에 GC 비율이 **극단적으로 낮은 read**(하위 N%)와 **높은 read**(상위 N%)를 골라, 그 read 들의 평균 품질을 **전체 평균 품질과 비교**한 요약을 출력한다.
- GC 비율은 lab07/src/fasta.py 의 gc_content 함수로 계산한다.
- 표준 라이브러리만 사용하므로 별도 설치 없이 실행된다.

2. 사용법

lab11 폴더 안에서 실행한다.

```
cd lab11
python3 src/gc_content.py <FASTQ 파일 경로> [극단 비율(%)]
```

인자	필수	설명
FASTQ 파일 경로	예	분석할 FASTQ 파일
극단 비율(%)	아니오	GC 하위·상위 몇 % 를 극단으로 볼지. 생략하면 5

FASTQ 파일을 주지 않으면 사용법을 안내하고 종료한다.

2-1. 실행 예시 (권장: 표는 파일로 저장)

read 마다 한 줄씩 출력되므로 read 가 20만 개면 표도 20만 줄이다. **표는 > 로 파일에 저장**하는 것이 좋다.

```
python3 src/gc_content.py data/SRR1553607_1.fastq > doc/gc_quality_1.tsv
```

요약 비교는 **표준 에러**(stderr)로 출력되기 때문에 > 로 표를 저장해도 화면에 그대로 보인다.

```
===== GC 비율 극단 read 의 평균 품질 비교 =====
전체 read 수: 203445
전체 평균 GC(%): 58.84
전체 평균 품질: 33.45

구분          read 수       GC 범위(%)    평균품질      전체와 차이
GC 하위 5%    10172         12.87~39.60   36.67         +3.22
GC 상위 5%    10172         73.27~98.02   30.83         -2.62
```

저장된 표(doc/gc_quality_1.tsv)의 앞부분:

```
이름          길이          GC(%)         평균품질
SRR1553607.1  101           42.57         37.44
SRR1553607.2  101           46.53         36.62
```

2-2. 극단 비율 바꾸기

더 극단적인 read 만 보려면 두 번째 인자로 비율을 준다.

```
python3 src/gc_content.py data/SRR1553607_1.fastq 1 > doc/gc_quality_1.tsv
GC 하위 1%    2034          12.87~30.69   37.53         +4.08
GC 상위 1%    2034          78.22~98.02   29.02         -4.43
```

3. 출력 항목 설명

3-1. read 별 표 (표준 출력)

열	설명

이름	FASTQ 1번째 줄에서 @ 를 뗀 첫 단어
길이	서열 길이 (bp)
GC(%)	(G 개수 + C 개수) ÷ 길이 × 100. 소문자도 대문자로 바꿔 센다
평균품질	그 read 품질 점수(Phred+33)의 평균

3-2. 요약 비교 (표준 에러)

항목	설명
GC 하위 N%	GC 비율로 정렬했을 때 가장 낮은 쪽 N% 의 read
GC 상위 N%	GC 비율로 정렬했을 때 가장 높은 쪽 N% 의 read
GC 범위(%)	그 그룹에 속한 read 의 GC 최솟값~최댓값
전체와 차이	그룹 평균품질 − 전체 평균품질 (+ 는 전체보다 높음)

극단을 고정값(예: 30% 미만, 70% 이상)이 아니라 비율(하위·상위 N%)로 정하는 이유: 이 시료는 전체 평균 GC 가 약 59% 로 높은 편이라, 고정값을 쓰면 "30% 미만" 은 약 1,600개, "70% 이상" 은 약 24,500개로 두 그룹 크기가 크게 달라진다. 비율로 정하면 시료마다 GC 분포가 달라도 양쪽 그룹 크기가 같아 비교하기 쉽다.

4. 결과: GC 극단 read 의 품질 비교

세 파일에 대해 기본값(하위·상위 5%)으로 실행한 결과이다.

파일	전체 평균품질	GC 하위 5%	GC 상위 5%
SRR1553607_1 (read 1)	33.45	36.67 (+3.22)	30.83 (−2.62)
SRR1553607_2 (read 2)	30.83	34.80 (+3.97)	23.64 (**−7.19**)
trimmed (트리밍 후)	34.85	37.27 (+2.43)	33.60 (−1.25)

해석

- **GC 가 높은 read 는 품질이 전체보다 낮고, GC 가 낮은 read 는 오히려 품질이 높았다.** 세 파일 모두 같은 경향이다.
- 차이는 **read 2 에서 가장 크다 (−7.19).** read 2 는 read 1 보다 시퀀싱 후반에 읽히므로 전체적으로 품질이 낮고, GC 가 높은 구간에서 더 크게 떨어진다.
- GC 가 99% 에 가까운 read 를 직접 보면 CCCCCCCC…, GGGGGGGG… 처럼 **G·C 가 길게 반복되고 평균 품질이 2~4** 인 경우가 많다. 이는 실제 유전체 서열이라기보다 **시퀀싱 오류로 생긴 인공 서열**일 가능성이 높다.
- GC 가 높은 서열은 DNA 이중 가닥이 단단하게 붙어 있고 2차 구조를 만들기 쉬워, PCR 증폭과 시퀀싱 반응이 고르게 일어나지 않는다. 이것이 **GC 편향(GC bias)** 으로 알려진 현상이다.
- **트리밍 후에는 차이가 −2.62 → −1.25 로 줄었다.** 품질이 낮은 끝부분과 read 가 제거되면서 GC 가 높은 저품질 read 도 많이 걸러졌기 때문이다.

5. 코드 구조

함수	하는 일
gc_content(seq)	(lab07 fasta.py) 서열의 GC 비율을 0~1 사이 값으로 돌려준다

mean_quality(qual)	품질 문자열의 평균 Phred 점수를 돌려준다
average(numbers)	숫자 리스트의 평균을 돌려준다
main()	FASTQ 를 한 줄씩 읽어 read 별 표를 출력하고, GC 순으로 정렬해 극단 그룹을 비교한다

다른 lab 폴더의 fasta.py 를 쓰기 위해 스크립트 위치 기준으로 lab07/src 를 sys.path 에 추가한다. 따라서 저장소 폴더 구조(lab07/src/fasta.py)가 그대로 있어야 한다.

6. 테스트

결과를 손으로 계산할 수 있는 작은 FASTQ 를 만들어 확인했다.

```
@r1 desc      GGCC  IIII   -> GC 100%, 품질 40
@r2           AATT  ####   -> GC   0%, 품질 2
@r3           acgt  5555   -> GC  50%, 품질 20  (소문자 서열)
@r4           ACGN  I#5?   -> GC  50%, 품질 (40+2+20+30)/4 = 23
python3 src/gc_content.py test.fq 25
이름           길이          GC(%)         평균품질
r1             4             100.00        40.00
r2             4             0.00          2.00
r3             4             50.00         20.00
r4             4             50.00         23.00

전체 평균 품질: 21.25
GC 하위 25%    1             0.00~0.00     2.00          -19.25
GC 상위 25%    1             100.00~100.00 40.00         +18.75
```

모든 값이 손 계산과 일치했다. 또한 실제 데이터의 **전체 평균 품질(33.45, 30.83)은** src/qual_summary.py **의 결과와 같았다.**

7. 주의할 점

- **Phred+33 전용**이다.
- FASTQ 가 **정확히 4줄에 read 하나**인 형식이라고 가정한다. .fastq.gz 는 먼저 압축을 풀어야 한다.
- N 은 GC 로 세지 않지만 길이에는 포함된다. N 이 많은 read 는 GC 비율이 실제보다 낮게 나온다.
- 요약 비교를 위해 read 마다 (GC, 평균품질) 두 숫자를 메모리에 모아 둔다. 수백만 read 정도는 문제없지만 아주 큰 파일에서는 메모리를 더 쓴다.
- 길이가 0 인 read 는 건너뛴다.